Algorithms and Data Structures

REPORT LAB 1

Name: ILGIN
Surname: CEYLAN
Student ID: 82862

TASK 1 : Implementation of Algorithms

I have implemented three functions with different time complexities as requested:

1. **Linear Search:** O(n) complexity.
2. **Insertion Sort:** O(n^2) complexity.
3. **Binary Search:** O(\log n) complexity.

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int linearSearch(int arr[], int n, int key) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == key) return i;
    }
    return -1;
}

void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n - 1, mid;
    while (low <= high) {
        mid = (low + high) / 2;
        if (arr[mid] == key) return mid;
        else if (arr[mid] < key) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}

void fillArray(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        arr[i] = rand() % (n * 10);
```

```
    }
}

int main(void) {
    int testCases[] = { 1000, 5000, 10000, 50000, 100000, 500000,
1000000 };
    int numTests = sizeof(testCases) / sizeof(testCases[0]);
    srand((unsigned int)time(0));

    printf("Algorithm Runtime Comparison\n\n");

    for (int i = 0; i < numTests; i++) {
        int n = testCases[i];
        int* arr = malloc(n * sizeof(int));
        fillArray(arr, n);
        int key = arr[n - 1];

        clock_t start, end;
        double time_used;

        // --- LINEAR SEARCH ---
        start = clock();
        linearSearch(arr, n, key);
        end = clock();
        time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
        printf("Linear Search (n=%d): %.6f sec\n", n, time_used);

        // --- INSERTION SORT ---
        if (n <= 100000) {
            start = clock();
            insertionSort(arr, n);
            end = clock();
            time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
            printf("Insertion Sort (n=%d): %.6f sec\n", n, time_used);
        } else {
            printf("Insertion Sort (n=%d): Skipped (Too slow)\n", n);
        }

        // --- BINARY SEARCH ---
        start = clock();
        binarySearch(arr, n, key);
        end = clock();
        time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
```

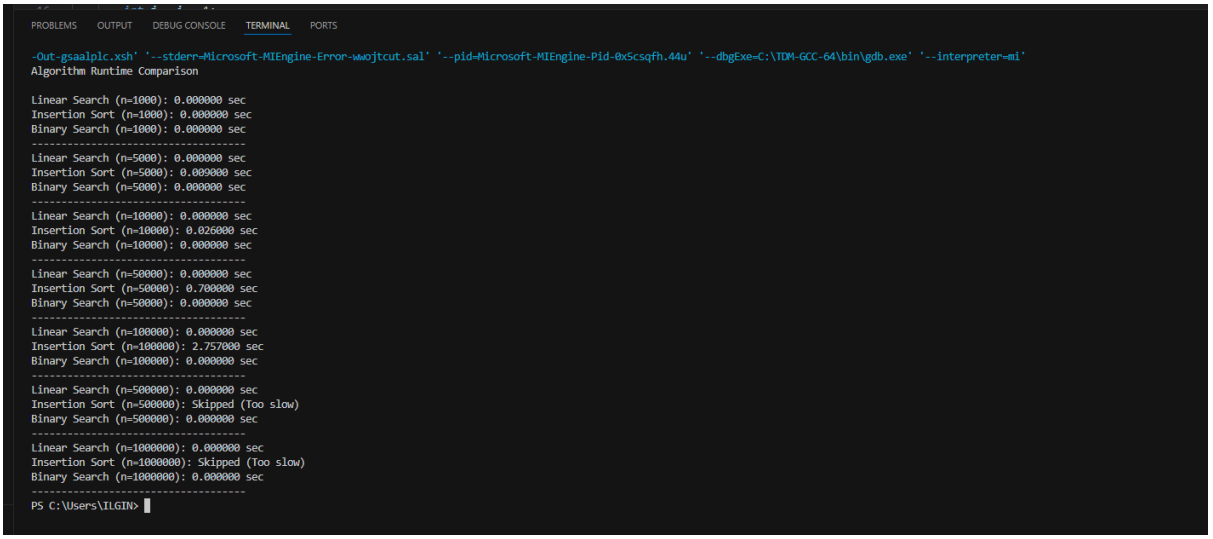
```
        printf("Binary Search (n=%d): %.6f sec\n", n, time_used);

        printf("-----------------------------------\n");
        free(arr);
    }
    return 0;
}
```

TASK 2 : Experimental Results (Runtime Table)

The following table shows the execution time (in seconds) for each input size (n):

n	Linear Search (O(n))	Insertion Sort (O(n2))	Binary Search (O(logn))
1000	0.000000	0.000000	0.000000
5000	0.000000	0.009000	0.000000
10000	0.000000	0.026000	0.000000
50000	0.000000	0.700000	0.000000
100000	0.000000	2.757000	0.000000
500000	0.000000	N/A (Too slow)	0.000000
1000000	0.000000	N/A (Too slow)	0.000000



TASK 3 : Interpretation of Results

The relationship between input size (n) and runtime growth is interpreted as follows:

- **Insertion Sort (O(n^2)):** As expected, this algorithm shows a quadratic growth rate. When the input size increases by 10 times (e.g., from 10,000 to 100,000), the runtime increases significantly (from 0.026s to 2.757s). For $n \ge 500,000$, the execution becomes impractical for real-time use.
- **Linear Search (O(n)):** This algorithm follows a linear growth pattern. However, due to modern CPU speeds, the execution time for the tested ranges remains near 0.000000 seconds, showing it is efficient for moderate datasets.
- **Binary Search (O(\log n)):** This is the most efficient algorithm. Even as n grows from 1,000 to 1,000,000, the runtime remains constant at 0.000000s, proving that logarithmic growth is highly scalable for large data.